



DEPLOYMENT RUNBOOK

Securelytix Dev SDK

PII Tokenization for Kubernetes

Field	Value
Version	2.0
Status	Testing Phase
Published Date	May 2026
Audited by	Securelytix DevOps
Target Audience	Confidential
SDK Version	1.0.4

⚠ WARNING - Confidential

This document contains private repository credentials and access tokens.
Do not share beyond the intended recipient.

Table of Contents

1. Overview	5
1.1 How It Works	5
1.2 Architecture	5
1.3 Scope and Limitations	6
1.4. Supported PII Fields	6
2. Quick Start	7
Step 1: Get your API key	7
Step 2: Add the Helm repo	7
Step 3: Create the DockerHub pull secret	7
Step 4: Install	7
Step 5: Port Forwarding	7
Step 6: Verify	7
3. Prerequisites	8
3.1 Kubernetes Cluster Requirements	8
3.2 Local Tools	8
3.3 Credentials and Access	8
4. Getting Your API Key	9
4.1 Steps	9
4.2 API Key Format	9
5. Local Testing with Docker	9
5.1 Start PostgreSQL Locally	9
5.2 Authenticate with DockerHub	10
5.3 Run the SDK Container	10
5.4 Verify	10
5.5 Test Tokenization End-to-End	10
5.6 Cleanup	11
6. Kubernetes Deployment	11
6.1 Interactive Installer (Recommended When Available)	11
6.2 Manual Deployment	11
Step 1: Add the Helm Repository	11
Step 2: Create the DockerHub Pull Secret	12
Step 3a: Deploy with Bundled PostgreSQL (Recommended for Testing)	12
Step 3b: Deploy with External PostgreSQL	12
Step 4: Verify the Deployment	12
6.3 Calling the SDK from Other Pods	13
6.4 Windows Deployment Guide	13
7. Configuration Reference	13
7.1 Helm Parameters	13

8. SDK Usage in Code	14
8.1 What to Change in Your Project	14
8.2 Where to Use the Vault API	14
HTTP / API Handlers	14
Runtime Tokenization - When to Call	14
Runtime Detokenization - When to Call	14
Storing Tokens Instead of Raw Values	15
8.3 Reference HTTP Clients	15
TypeScript / JavaScript	15
Go	16
8.4 Minimal Code Patterns	17
Ingest / Create - Tokenize, then Persist	17
Privileged Read - Load, Detokenize, Return	17
8.5 TypeScript / JavaScript - API Route Examples	17
Incoming POST /api/users - Tokenize Before Persisting	17
Outgoing GET /api/admin/users/:id - Detokenize Before Response	18
8.6 Go - API Handler Examples	18
Incoming POST - Tokenize After Decode, Before INSERT	18
Privileged GET - Detokenize After SELECT, Before Encode	18
9. API Reference	19
9.1 POST /tokenize	19
Request	19
Response - 200 OK	19
Error Responses	19
9.2 POST /detokenize	20
Request	20
Response - 200 OK	20
9.3 GET /health	20
10. Database Access	21
10.1 Bundled PostgreSQL Credentials	21
10.2 Connecting via Port-Forward	21
10.3 Retrieving a Customized Password	21
10.4 Backup and Restore	21
11. Health Checks and Troubleshooting	22
11.1 Health Check	22
11.2 Common Issues	22
Issue: Image pull error (ErrImagePull / ImagePullBackOff)	22
Issue: zsh: no matches found: imagePullSecrets[0].name	22
Issue: PostgreSQL connection failed / password authentication failed for user 'admin'	23
Issue: Pod stuck in Pending state	23
Issue: 401 Unauthorized on /tokenize	23

Issue: 429 Too Many Requests	23
11.3 Useful Diagnostic Commands	23
12. Performance and Benchmarks	24
12.1 Expected Performance (Dev SDK v1.0.4)	24
12.2 Scaling Recommendations	24
12.3 What Securelytix Does NOT Do	24
13. Uninstall	25
13.1 Interactive Uninstaller	25
13.2 Manual Uninstall	25
13.3 What Gets Removed	25
13.4 What is Preserved	25
14. Support and Contact	25
14.1 Reporting Issues	26
15. Quick Reference Card	26
Get your API key	26
Test locally (Docker)	26
Deploy to Kubernetes	26
Tokenize	27
Detokenize	27
Common diagnostic commands	27
Support	27

Quick Links:

- I just want to test locally → Section 5
- I want to deploy to Kubernetes → Section 6
- I want to integrate the SDK in my code → Section 8
- Show me the API endpoints → Section 9

1. Overview

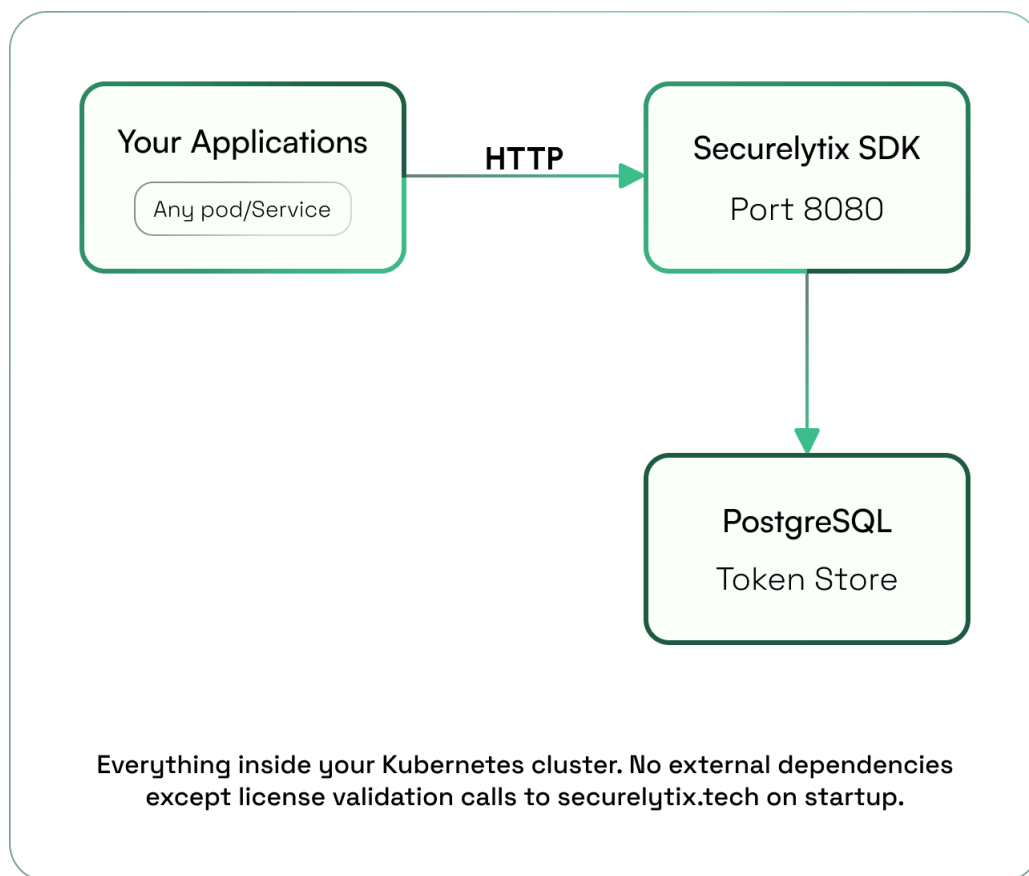
The Securelytix Dev SDK is a PII tokenization service that runs on Kubernetes. It detects sensitive personal data - names, emails, phone numbers, credit cards, and similar - and replaces them with secure, format-preserving tokens. All services downstream of the SDK receive only tokens; the actual sensitive data stays in the vault.

1.1 How It Works

1. Your application sends a sensitive value (such as an email address) to the SDK.
2. The SDK stores the real value securely in PostgreSQL and returns a format-preserving token.
3. Downstream services, logs, and analytics receive only the token - never the plaintext value.
4. When the original value is genuinely needed, your trusted service calls the SDK to detokenize.

1.2 Architecture

The Dev SDK runs as a single container in your Kubernetes cluster, backed by PostgreSQL. The SDK exposes a simple HTTP API. Your applications call the SDK over the cluster's internal network - sensitive data never leaves your VPC.



1.3 Scope and Limitations

Here is the list which is supported by version 1.0.4 dev-sdk

Capability	Supported
Structured data (JSON fields, DB columns, form inputs)	Yes
Common PII types (email, phone, credit card, name)	Yes
Format-preserving tokenization	Yes
Reversible tokenization (detokenize)	Yes
File tokenization	Upcoming Feature
Streaming data	Upcoming Feature
Production-grade KMS integration	Upcoming Feature
Multi-tenancy / team isolation	No - single tenant

i INFO - Dev SDK is for testing only

This is the developer / evaluation tier. For production deployments with KMS-backed encryption, audit logging, and SLA-backed support, contact us about the Paid tier.

1.4. Supported PII Fields

The vault automatically detects and tokenizes the following PII types. Fields not matching any pattern are passed through unchanged and tagged as unknown.

PII Type	Supported
Full Name	Yes
Email Address	Yes
Phone Number	Yes
PAN Card	Yes
Aadhaar	Yes
UPI ID	Yes
Date of Birth	Yes
Credit / Debit Card	Yes
Age	Yes
Gender	Yes

IP Address	Yes
Unknown / Non-PII	No

2. Quick Start

If you want to be up and running in 10 minutes on Kubernetes, follow these five steps. Each is explained in full detail in later sections - this is the speed-run version.

Step 1: Get your API key

Visit securelytix.tech, click Get API Key, enter your work email. Copy the key (begins with sk_dev_).

Step 2: Add the Helm repo

```
helm repo add securelytix https://charts.securelytix.tech
helm repo update
```

Step 3: Create the DockerHub pull secret

```
kubectl create secret docker-registry securelytix-dockerhub \
  --docker-server=https://index.docker.io/v1/ \
  --docker-username=securelytix2026 \
  --docker-password=<YOUR_PAT_TOKEN>
```

Step 4: Install

```
helm install dev-sdk securelytix/dev-sdk \
  --set secrets.apiKey="<your-api-key>" \
  --set postgresql.enabled=true \
  --set "imagePullSecrets[0].name=securelytix-dockerhub"
```

⚠ WARNING - mac users

The imagePullSecrets argument must always be wrapped in double quotes as shown above. Unquoted square brackets cause a glob error in mac(zsh). bash users are not affected.

Step 5: Port Forwarding

```
kubectl port-forward svc/dev-sdk 8080:8080
```

Step 6: Verify

```
curl http://localhost:8080/health
# Expected: {"status":"ok","timestamp":"...","postgres":"ok"}
```

✓ TIP - That's it

Your SDK is now running. Jump to Section 8 for the API reference, Section 9 for code examples, or Section 11 if anything went wrong.

3. Prerequisites

3.1 Kubernetes Cluster Requirements

- Kubernetes 1.20 or later - 1.24+ recommended
- 2+ CPU cores available for the SDK and PostgreSQL combined
- 4 GB RAM minimum (8 GB recommended for production-like testing)
- 10 GB storage for PostgreSQL (bundled) or access to an external PostgreSQL instance
- Outbound HTTPS access from the cluster to: hub.docker.com (image pull), securelytix.tech (license validation), index.docker.io (Docker auth)

3.2 Local Tools

Tool	Version	Purpose
Helm	3.0+	Package manager for Kubernetes
kubectl	1.20+	Cluster management CLI
Docker	Any recent	Local testing only (not needed for K8s deployment)

3.3 Credentials and Access

You will need the following credentials. Items marked 'Shared separately' will be sent over secure channels by the Securelytix team.

Credential	Source	Required for
Securelytix API Key	Self-serve at securelytix.tech	License validation
DockerHub Username	securelytix2026	Image pull
DockerHub PAT Token	s.support@securelytix.tech	Private image access
GitHub Repo Access	github.com/Securelytix-dev/securelytix	Code review (optional)

⚠ WARNING - Private DockerHub image

The Docker image is hosted on a private DockerHub repository. You must create a Kubernetes pull secret before deploying. Section 6 covers this step.

4. Getting Your API Key

A Securelytix API key is required before you deploy or use the SDK. Keys are provisioned via the website in a self-serve flow - no procurement delays, no sales calls.

4.1 Steps

5. Visit <https://securelytix.tech>
6. Click the 'Get API Key' button in the top-right corner of the homepage (marked with a 'New' badge)
7. On the landing step: enter your enterprise work email address (for example, `dev@example.com`), tick the agreement checkbox, and click 'Get my API key →'
8. Complete email verification and basic account setup
9. Your API key is provisioned immediately after verification. Copy it and store it securely - you will need it during SDK installation

⚠ **WARNING - Enterprise email required**

Personal email providers (Gmail, Hotmail, Yahoo, Outlook.com) are not accepted. Use your organization email, for example `dev@idify.com`.

4.2 API Key Format

Your API key looks like this:

```
sdGxOV4tnJAKsQj--BYhW8Fh6BhPtEmcW0SLsNXA1YALiaWGdYdUvfRbamGRFdCJ
```

x CRITICAL - Keep your API key safe

Never commit your API key to source control. Never share it in chat or email. Always pass it through a Kubernetes Secret or your secrets manager at deploy time. If you suspect a compromise, rotate it immediately from the dashboard.

5. Local Testing with Docker

For a quick sanity check without spinning up a Kubernetes cluster, you can run the SDK on Docker against a local PostgreSQL container. This is the fastest path to confirming the SDK works in your environment.

5.1 Start PostgreSQL Locally

```
docker run -d \  
  --name securelytix-postgres \  
  -e POSTGRES_USER=vault \  
  -e POSTGRES_PASSWORD=vault
```

```
-e POSTGRES_PASSWORD=vault \
-e POSTGRES_DB=vault \
-p 5432:5432 \
postgres:15
```

5.2 Authenticate with DockerHub

The SDK image is in a private repository. Log in before pulling:

```
docker login -u securelytix2026
# Enter your PAT token when prompted (not a password)
```

5.3 Run the SDK Container

```
docker run -d \
  --name securelytix-sdk \
  -p 8080:8080 \
  -e VAULT_PORT=8080 \
  -e
DATABASE_URL="postgresql://vault:vault@host.docker.internal:5432/vault?sslmode=disable" \
  -e API_KEY="<your-api-key>" \
  securelytix2026/dev-sdk:1.0.4
```

5.4 Verify

Check that the SDK is healthy:

```
curl http://localhost:8080/health

# Expected response:
{"status":"ok","timestamp":"2026-05-15T10:30:00Z","postgres":"ok"}
```

5.5 Test Tokenization End-to-End

Tokenize a sample email:

```
curl -sS -X POST "http://localhost:8080/api/v1/tokenize" \
  -H "Content-Type: application/json" \
  -d '{
    "data": {
      "email": "user@example.com",
      "name": "user example"
    }
  }'

# Expected response:
{"data":
{"email":"gfhw@reviudg.cnr\_stx",
"Name":"vpqghbmudaon_stx"
},
"Request_id":"e1a94539-8ae9-4538-8329-999d27765a19",
"latency":"19.33925ms"}
```

Note - Send tokenized values in detokenize request to get the original value

Detokenize it back:

```
curl -sS -X POST "http://localhost:8080/api/v1/detokenize" -H "Content-Type: application/json" -d '{"data":{"email":"bpst@gbjsnko.jot_stx","name":"pgkgpgrircg_stx"}}'
```

Expected response:

```
{
  "data":
  {"email":"user@example.com",
   "name":"user example"},
  "Status":"success",
  "Request_id":"322ce6bd-d400-473b-b523-cdbf689172bb",
  "Latency":"49.586375ms"
}
```

5.6 Cleanup

```
docker stop securelytix-sdk securelytix-postgres
docker rm securelytix-sdk securelytix-postgres
```

6. Kubernetes Deployment

6.1 Interactive Installer (Recommended When Available)

```
curl -fsSL https://charts.securelytix.tech/install.sh | bash
```

The installer guides you through API key entry, database choice, namespace selection, and pull secret setup. This script will become available once the GitHub repository is made public.

WARNING - Interactive installer not yet available

The interactive installer script is not yet available during this testing phase. Please use the Manual Deployment steps in Section 6.2 below.

6.2 Manual Deployment

Step 1: Add the Helm Repository

```
helm repo add securelytix https://charts.securelytix.tech
helm repo update
```

Step 2: Create the DockerHub Pull Secret

The SDK image is private. Create a Kubernetes secret so the cluster can pull it:

```
kubectl create secret docker-registry securelytix-dockerhub \
  --docker-server=https://index.docker.io/v1/ \
  --docker-username=securelytix2026 \
  --docker-password=<YOUR_PAT_TOKEN>
```

NOTE - Custom namespace: If you are deploying to a specific namespace, append -n <namespace> to the kubectl command above, and to every kubectl command in the rest of this section.

Step 3a: Deploy with Bundled PostgreSQL (Recommended for Testing)

This is the fastest path. No external database required:

```
helm install dev-sdk securelytix/dev-sdk \
  --set secrets.apiKey="<your-api-key>" \
  --set postgresql.enabled=true \
  --set "imagePullSecrets[0].name=securelytix-dockerhub"
```

⚠ WARNING - mac users

The imagePullSecrets argument must always be wrapped in double quotes as shown above. Unquoted square brackets cause a glob error in mac(zsh). bash users are not affected.

Step 3b: Deploy with External PostgreSQL

If you already have a PostgreSQL instance you want to use:

```
helm install dev-sdk securelytix/dev-sdk \
  --set secrets.apiKey="<your-api-key>" \
  --set
secrets.databaseUrl="postgresql://user:pass@host:5432/dbname?sslmode=disable" \
  --set "imagePullSecrets[0].name=securelytix-dockerhub"
```

Step 4: Verify the Deployment

Check that everything is up:

```
# Check pods are running
kubectl get pods -l app.kubernetes.io/instance=dev-sdk

# Check the service has a cluster IP
kubectl get svc dev-sdk

# Check logs for any startup errors
kubectl logs -l app.kubernetes.io/instance=dev-sdk --tail=20

# Port-forward and verify the health endpoint
kubectl port-forward svc/dev-sdk 8080:8080 &
curl http://localhost:8080/health
# Expected: {"status":"ok","timestamp":"...", "postgres":"ok"}
```

6.3 Calling the SDK from Other Pods

Once deployed, your application pods can reach the SDK at its internal cluster DNS:

```
# If deployed in the default namespace:  
http://dev-sdk.default.svc.cluster.local:8080  
  
# If deployed in a custom namespace (e.g. 'tokenization'):  
http://dev-sdk.tokenization.svc.cluster.local:8080
```

This means application code running in any pod in the cluster can call the SDK without exposing it externally. See Section 9 for SDK usage in code.

6.4. Windows Deployment Guide

6.4.1. Prerequisites

Install the following tools before proceeding. All commands in this guide are written for PowerShell.

Tool	Version	Install
Docker Desktop	Any recent	https://www.docker.com/products/docker-desktop/
kubectl	1.20+	https://kubernetes.io/docs/tasks/tools/install-kubectl-windows/
Helm	3.0+	winget install Helm.Helm

⚠ **PowerShell Required** - All commands in this guide use PowerShell syntax. If you are using Command Prompt (cmd.exe), replace backtick (`) line continuations with caret (^).

6.4.2. Kubernetes Setup (Docker Desktop)

Docker Desktop for Windows includes a built-in Kubernetes cluster. Enable it before proceeding:

- Open Docker Desktop → Settings → Kubernetes
- Check Enable Kubernetes
- Click Apply & Restart and wait for the green Kubernetes indicator

Verify kubectl is pointing at Docker Desktop:

```
kubectl config use-context docker-desktop  
kubectl get nodes
```

6.4.3. Add Helm Repository

```
helm repo add securelytix https://charts.securelytix.tech
helm repo update
```

6.4.4. Create DockerHub Pull Secret

The SDK image is on a private DockerHub repository. Run this once per cluster:

```
kubectl create secret docker-registry securelytix-dockerhub `
--docker-server=https://index.docker.io/v1/ `
--docker-username=securelytix2026 `
--docker-password=<YOUR_PAT_TOKEN>
```

NOTE - Replace <YOUR_PAT_TOKEN> with the DockerHub PAT shared by the Securelytix team.

6.4.5. Install via Helm

6.4.5.(a) With Bundled PostgreSQL (Recommended for Testing)

No external database required - fastest path to get started:

```
helm install dev-sdk securelytix/dev-sdk `
--set secrets.apiKey="<your-api-key>" `
--set postgresql.enabled=true `
--set "imagePullSecrets[0].name=securelytix-dockerhub"
```

⚠ WARNING - Always wrap the imagePullSecrets argument in double quotes as shown. Unquoted square brackets will cause a parse error in PowerShell.

6.4.5.(b) With External PostgreSQL

```
helm install dev-sdk securelytix/dev-sdk `
--set secrets.apiKey="<your-api-key>" `
--set
secrets.databaseUrl="postgresql://user:pass@host:5432/dbname?sslmode=disable" `
--set "imagePullSecrets[0].name=securelytix-dockerhub"
```

6.4.6. Verify the Deployment

```
# Check pods are running
kubectl get pods -l app.kubernetes.io/instance=dev-sdk

# Check the service
kubectl get svc dev-sdk-dev-sdk

# Check startup logs
kubectl logs -l app.kubernetes.io/instance=dev-sdk --tail=20
```

6.4.7. Port Forward and Test

Start Port Forward

Open a PowerShell terminal and run:

```
kubectl port-forward svc/dev-sdk-dev-sdk 8080:8080
```

NOTE - Keep this terminal open. Open a second terminal for the curl commands below.

Health Check

```
curl.exe http://localhost:8080/health
```

Expected response:

```
{"status":"ok","timestamp":"...","postgres":"ok"}
```

Test Tokenization

```
curl.exe -sS -X POST "http://localhost:8080/api/v1/tokenize" `
-H "Content-Type: application/json" `
-d '{"data":{"email":"user@example.com","name":"user example"}}'
```

Test Detokenization

Use the tokens returned from the tokenize call above:

```
curl.exe -sS -X POST "http://localhost:8080/api/v1/detokenize" `
-H "Content-Type: application/json" `
-d '{"data":{"email":"<token>","name":"<token>"} }'
```

6.4.8. Upgrading

When a new image is released, upgrade with:

```
helm repo update securelytix
helm upgrade dev-sdk securelytix/dev-sdk --reuse-values
```

6.4.9. Uninstall

```
helm uninstall dev-sdk

# Also remove the pull secret if no longer needed
kubectl delete secret securelytix-dockerhub
```

6.4.10. Troubleshooting

Issue: imagePullSecrets parse error

```
zsh: no matches found: imagePullSecrets[0].name ← (not applicable on Windows)
```

This error does not occur in PowerShell. If you see a similar parse error, ensure the argument is wrapped in double quotes:

```
--set "imagePullSecrets[0].name=securelytix-dockerhub"
```

Issue: curl vs curl.exe

In PowerShell, curl is an alias for Invoke-WebRequest. Always use curl.exe to call the actual curl binary:


```
# Wrong - uses Invoke-WebRequest alias
curl http://localhost:8080/health

# Correct
curl.exe http://localhost:8080/health
```

Issue: Pod stuck in ImagePullBackOff

The DockerHub pull secret is missing or incorrect. Recreate it:

```
kubectl delete secret securelytix-dockerhub
kubectl create secret docker-registry securelytix-dockerhub `
  --docker-server=https://index.docker.io/v1/ `
  --docker-username=securelytix2026 `
  --docker-password=<YOUR_PAT_TOKEN>
```

Issue: partial_success on detokenize

The tokens were not found in the vault database. This usually means you tokenized on a different container or the database was reset. Tokenize first on your current running instance, then detokenize the tokens you just received.

Issue: service dev-sdk not found on port-forward

The correct service name is dev-sdk-dev-sdk (not dev-sdk). Use:

```
kubectl port-forward svc/dev-sdk-dev-sdk 8080:8080
```

7. Configuration Reference

All configuration is passed via Helm --set flags or a values.yaml override file.

7.1 Helm Parameters

Parameter	Description	Required	Default
secrets.apiKey	API key for license validation	Yes	-
secrets.databaseUrl	PostgreSQL connection string	If no bundled PG	-
postgresql.enabled	Enable bundled PostgreSQL	No	false
postgresql.auth.username	PostgreSQL username	No	vault
postgresql.auth.password	PostgreSQL password	No	vault
postgresql.auth.database	PostgreSQL database name	No	vault
config.port	Server port	No	8080
config.logLevel	Log level (info / debug / warn)	No	info
config.requestLimit	Max requests per second per API key	No	25
config.tokenTtl	Token expiration in seconds (0 = never)	No	0
auditLog.enabled	Enable audit logging to DB	No	false
auditLog.retentionDays	Audit log retention in days	No	30
imagePullSecrets	DockerHub pull secret name	Yes	-
persistence.enabled	Enable persistent storage	No	true
persistence.size	Storage size	No	1Gi
resources.requests.cpu	CPU request	No	100m
resources.requests.memory	Memory request	No	256Mi
resources.limits.cpu	CPU limit	No	500m
resources.limits.memory	Memory limit	No	512Mi

8. SDK Usage in Code

This section covers approaches to integrating the Securelytix SDK: the lower-level HTTP client pattern for teams that prefer to own their integration directly. These approaches follow the same write path (tokenize before persisting) and read path (detokenize only in trusted services).

8.1 What to Change in Your Project

Two steps are all that is required to integrate the SDK into an existing application:

Step	Action
1	Add VAULT_URL (or equivalent) to your app config / secrets.
2	Add a small HTTP client with two methods: tokenize and detokenize (see section 9.3).

Integration is HTTP + JSON only. You do not need to add this repository as a Go module.

8.2 Where to Use the Vault API

Use the vault anywhere your code touches PII-shaped JSON. The common pattern is runtime tokenization on the way in (before persistence or fan-out) and runtime detokenization only when a trusted path truly needs plaintext.

HTTP / API Handlers

- Incoming API calls (create / update / submit): Build the same data object you would have saved before. Call tokenize in the request handler, then store tokens in your database instead of raw values.
- Outgoing API responses: Often return stored tokens as-is. For admin, support, or internal endpoints that must return real values, call detokenize after loading the row - right before serializing the response.
- Downstream HTTP calls: Before you POST to another microservice or webhook, tokenize the JSON body so partners and logs see tokens, not raw PII.

Runtime Tokenization - When to Call

Call tokenize at runtime whenever a value is about to leave the short-lived, trusted zone: before INSERT/UPDATE, before enqueueing a job, before writing to a cache key that might be dumped, or before emitting to a bus.

Runtime Detokenization - When to Call

Call detokenize at runtime only in narrow flows: reporting export, fulfillment, billing reconciliation, or a screened internal UI. Avoid detokenizing on every read if the consumer only needs the tokenized view.

Storing Tokens Instead of Raw Values

After tokenization, the response data has the same keys and nesting as your input; only classified PII fields become tokens. Persist that object. Logs, replicas, and backups of your DB are less exposed to raw PII as long as you never persist the pre-tokenize payload.

8.3 Reference HTTP Clients

Copy the appropriate client into your repo and wire once at startup. These lightweight clients require no additional dependencies beyond a standard HTTP library.

TypeScript / JavaScript

```

export type VaultClientOptions = { baseUrl: string; fetchImpl?: typeof fetch };

export function createVaultClient({ baseUrl, fetchImpl = fetch }:
VaultClientOptions) {
  const root = baseUrl.replace(/\/$/, "");

  async function tokenize(
    data: Record<string, unknown>,
    headers?: { customerId?: string }
  ): Promise<{ data: Record<string, unknown>; request_id: string }> {
    const r = await fetchImpl(`${root}/api/v1/tokenize`, {
      method: "POST",
      headers: {
        "Content-Type": "application/json",
        ...(headers?.customerId ? { "X-Customer-ID": headers.customerId } : {}),
      },
      body: JSON.stringify({ data }),
    });
    if (!r.ok) throw new Error(`vault tokenize ${r.status}: ${await r.text()}`);
    return r.json();
  }

  async function detokenize(data: unknown): Promise<{
    data: unknown;
    status: string;
    failed_fields?: { field: string; token: string }[];
    request_id: string;
  }> {
    const r = await fetchImpl(`${root}/api/v1/detokenize`, {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ data }),
    });
    if (!r.ok) throw new Error(`vault detokenize ${r.status}: ${await r.text()}`);
    return r.json();
  }

  return { tokenize, detokenize };
}

```

Wire once at startup:

```
const vault = createVaultClient({ baseUrl: process.env.VAULT_URL! });
```

Go

```

package vaultclient
import (
    "bytes"
    "context"
    "encoding/json"
    "fmt"
    "net/http"
    "strings"
)

```

```

type Client struct {
    BaseURL string
    HTTP     *http.Client
}
func New(baseURL string) *Client {
    return &Client{BaseURL: strings.TrimRight(baseURL, "/"), HTTP:
http.DefaultClient}
}
func (c *Client) Tokenize(ctx context.Context, data map[string]any, customerID
string) (map[string]any, error) {
    body, _ := json.Marshal(map[string]any{"data": data})
    req, err := http.NewRequestWithContext(ctx, http.MethodPost,
c.BaseURL+"/api/v1/tokenize", bytes.NewReader(body))
    if err != nil {
        return nil, err
    }
    req.Header.Set("Content-Type", "application/json")
    if customerID != "" {
        req.Header.Set("X-Customer-ID", customerID)
    }
    resp, err := c.HTTP.Do(req)
    if err != nil {
        return nil, err
    }
    defer resp.Body.Close()
    if resp.StatusCode < 200 || resp.StatusCode >= 300 {
        return nil, fmt.Errorf("tokenize: status %d", resp.StatusCode)
    }
    var out struct{ Data map[string]any }
    if err := json.NewDecoder(resp.Body).Decode(&out); err != nil {
        return nil, err
    }
    return out.Data, nil
}
func (c *Client) Detokenize(ctx context.Context, data map[string]any)
(map[string]any, bool, []string, error) {
    body, _ := json.Marshal(map[string]any{"data": data})
    req, err := http.NewRequestWithContext(ctx, http.MethodPost,
c.BaseURL+"/api/v1/detokenize", bytes.NewReader(body))
    if err != nil {
        return nil, false, nil, err
    }
    req.Header.Set("Content-Type", "application/json")
    resp, err := c.HTTP.Do(req)
    if err != nil {
        return nil, false, nil, err
    }
    defer resp.Body.Close()
    if resp.StatusCode < 200 || resp.StatusCode >= 300 {
        return nil, false, nil, fmt.Errorf("detokenize: status %d",
resp.StatusCode)
    }
    var out struct {
        Data          map[string]any `json:"data"`
        Partial        bool          `json:"partial"`
        FailedFields []string      `json:"failed_fields"`
    }
    if err := json.NewDecoder(resp.Body).Decode(&out); err != nil {
        return nil, false, nil, err
    }
    return out.Data, out.Partial, out.FailedFields, nil
}

```

8.4 Minimal Code Patterns

Ingest / Create - Tokenize, then Persist

```
const { data } = await vault.tokenize(payload, { customerId: tenantId });
await saveToDatabase({ ...record, payload: data });
```

Privileged Read - Load, Detokenize, Return

```
const row = await loadFromDatabase(id);
const { data, status, failed_fields } = await vault.detokenize(row.payload);
if (status === "partial_success" && failed_fields?.length) {
  /* log metrics; some fields may remain token strings */
}
return data;
```

8.5 TypeScript / JavaScript - API Route Examples

Incoming POST /api/users - Tokenize Before Persisting

```
app.post("/api/users", async (req, res) => {
  const body = req.body as Record<string, unknown>;

  // ADD: tokenize before persisting raw PII
  const { data: safe } = await vault.tokenize(body, {
    customerId: req.header("x-tenant-id") ?? undefined,
  });

  await saveUser(safe); // CHANGE: pass `safe` (tokens), not `body`
  res.status(201).json({ id: "..." });
});
```

Outgoing GET /api/admin/users/:id - Detokenize Before Response

```
app.get("/api/admin/users/:id", requireAdmin, async (req, res) => {
  const row = await loadUser(req.params.id); // holds tokens in DB

  // ADD: detokenize only for this privileged response
  const { data, status, failed_fields } = await vault.detokenize(row.profile);
  if (status === "partial_success" && failed_fields?.length) {
    req.log?.warn({ failed_fields }, "partial detokenize");
  }

  res.json({ ...row, profile: data }); // CHANGE: return plaintext for admin
});
```

8.6 Go - API Handler Examples

Incoming POST - Tokenize After Decode, Before INSERT

```
func (s *Server) createUser(w http.ResponseWriter, r *http.Request) {
    ctx := r.Context()
    var body map[string]any
    if err := json.NewDecoder(r.Body).Decode(&body); err != nil { return }

    // ADD: tokenize before persisting
    safe, err := s.vault.Tokenize(ctx, body, //Optional
r.Header.Get("X-Tenant-ID"))
    if err != nil { http.Error(w, "tokenize", http.StatusBadGateway); return }

    if err := s.store.InsertUser(ctx, safe); err != nil { return }
    w.WriteHeader(http.StatusCreated)
}
```

Privileged GET - Detokenize After SELECT, Before Encode

```
func (s *Server) adminGetUser(w http.ResponseWriter, r *http.Request) {
    ctx := r.Context()
    row, err := s.store.GetUser(ctx, id) // row.Profile has tokens
    if err != nil { return }

    // ADD: detokenize only for this handler
    plain, status, failed, err := s.vault.Detokenize(ctx, row.Profile)
    if err != nil { http.Error(w, "detokenize", http.StatusBadGateway); return }
    if status == "partial_success" && len(failed) > 0 {
        _ = failed // log or metric here
    }

    row.Profile = plain
    _ = json.NewEncoder(w).Encode(row)
}
```

Your JSON shape stays the same; only classified PII values become opaque tokens until you explicitly detokenize.

9. API Reference

All endpoints accept and return JSON. Authentication is via a Bearer token in the Authorization header. The Bearer token is your API key.

9.1 POST /tokenize

Convert a sensitive value into a format-preserving token.

Request

```
curl -sS -X POST "http://localhost:8080/api/v1/tokenize" \
```

```
-H "Content-Type: application/json" \  
-d '{  
  "data": {  
    "email": "user@example.com",  
    "name": "user example"  
  }  
'
```

Response - 200 OK

```
{  
  "data": {  
    "email": "qfhw@reviudq.cnr_stx",  
    "Name": "vpqhb mudaon_stx"  
  },  
  "Request_id": "e1a94539-8ae9-4538-8329-999d27765a19",  
}
```

Error Responses

Status	Reason	Body
400	Invalid value or unsupported type	{"error": "invalid_type", "message": "..."}
401	Missing or invalid API key	{"error": "unauthorized"}
429	Rate limit exceeded	{"error": "rate_limit", "retry_after": 1}
500	Internal error (DB, license)	{"error": "internal_error", "message": "..."}

9.2 POST /detokenize

Reverse a token back to its original plaintext value. Call this only from trusted services.

Request

```
curl -sS -X POST "http://localhost:8080/api/v1/detokenize" \  
-H "Content-Type: application/json" \  
-d '{  
  "data": {  
    "email": "qfhw@reviudq.cnr_stx",  
    "name": "vpqh bmudaon_stx"  
  }  
'
```

Response - 200 OK

```
{  
  "data":
```



```
{ "email": "user@example.com",  
  "name": "user example",  
  "Status": "success",  
  "Request_id": "322ce6bd-d400-473b-b523-cdbf689172bb",  
}
```

x CRITICAL - Detokenize is a trust boundary

Any service that holds the API key can detokenize every token in the vault.

Restrict detokenize-capable services to only those that genuinely need plaintext (back-office tools, customer support consoles).

Frontend and analytics pipelines should never have access.

9.3 GET /health

Health check endpoint. No authentication required. Use this for Kubernetes liveness/readiness probes.

```
{  
  "status": "ok",  
  "timestamp": "2026-05-15T10:30:00Z",  
  "postgres": "ok",  
  "version": "1.0.4"  
}
```

10. Database Access

Direct database access is useful for debugging, backups, and inspecting state. Avoid running ad-hoc queries in production.

10.1 Bundled PostgreSQL Credentials

If you deployed with postgresql.enabled=true and did not override credentials, defaults are:

Field	Value
Username	vault
Password	vault
Database	vault
Port	5432
Service name	dev-sdk-postgresql

10.2 Connecting via Port-Forward

The database runs inside the cluster. To connect a local SQL client (psql, PgAdmin, DBeaver):

```
# Default namespace:
kubectl port-forward svc/dev-sdk-postgresql 5432:5432

# Custom namespace:
kubectl port-forward svc/dev-sdk-postgresql 5432:5432 -n <namespace>
```

Then point your client at localhost:5432 with the credentials above. Keep the terminal open while connected.

10.3 Retrieving a Customized Password

```
# Default namespace:
kubectl get secret dev-sdk-postgresql -o jsonpath='{.data.password}' | base64 -d

# Custom namespace:
kubectl get secret dev-sdk-postgresql -n <namespace> -o jsonpath='{.data.password}' | base64 -d
```

10.4 Backup and Restore

Back up the bundled PostgreSQL:

```
kubectl exec dev-sdk-postgresql-0 -- pg_dump -U vault -d vault > backup.sql
```

Restore from a backup:

```
kubectl exec -i dev-sdk-postgresql-0 -- psql -U vault -d vault < backup.sql
```

11. Health Checks and Troubleshooting

11.1 Health Check

```
# Port-forward first
kubectl port-forward svc/dev-sdk 8080:8080

# In a separate terminal
curl http://localhost:8080/health

# Expected response
{"status":"ok","timestamp":"...", "postgres":"ok","version":"1.0.4"}
```

11.2 Common Issues

Issue: Image pull error (ErrImagePull / ImagePullBackOff)

Root cause: The DockerHub pull secret is missing, incorrect, or not referenced by the deployment.

Solution - Step 1: verify the secret exists:

```
kubectl get secret securelytix-dockerhub
```

If not found, recreate it:

```
kubectl create secret docker-registry securelytix-dockerhub \
  --docker-server=https://index.docker.io/v1/ \
  --docker-username=securelytix2026 \
  --docker-password=<YOUR_PAT_TOKEN>
```

Step 2 - confirm the deployment uses the secret:

```
kubectl get deployment dev-sdk \
  -o jsonpath='{.spec.template.spec.imagePullSecrets}'

# Should show: [{"name":"securelytix-dockerhub"}]
```

Step 3 - verify the image is pullable with your credentials:

```
docker login -u securelytix2026
docker pull securelytix2026/dev-sdk:1.0.4
```

Issue: zsh: no matches found: imagePullSecrets[0].name

Root cause: This is a zsh shell glob error. zsh interprets unquoted square brackets.

Always wrap the imagePullSecrets argument in double quotes:

```
--set "imagePullSecrets[0].name=securelytix-dockerhub"
```

Issue: PostgreSQL connection failed / password authentication failed for user 'admin'

Root cause: You are using the wrong username. The bundled PostgreSQL has user 'vault', not 'admin'.

```
psql postgresql://vault:vault@localhost:5432/vault
```

Issue: Pod stuck in Pending state

Root cause: Insufficient cluster resources, or a PersistentVolumeClaim that cannot be bound.

```
kubectl describe pod -l app.kubernetes.io/instance=dev-sdk
kubectl get pvc
kubectl describe nodes | grep -E 'Capacity|Allocated'
```

Issue: 401 Unauthorized on /tokenize

Root cause: The API key is missing, malformed, or invalid.

```
kubectl get secret dev-sdk-secrets \
-o jsonpath='{.data.api-key}' | base64 -d
```

Issue: 429 Too Many Requests

Root cause: Your traffic exceeds the configured rate limit (25 req/sec by default).

When API-key usage is exhausted, then also we are showing this 429

```
helm upgrade dev-sdk securelytix/dev-sdk \
--set config.requestLimit=200 --reuse-values
```

11.3 Useful Diagnostic Commands

Task	Command
List SDK pods	<code>kubectl get pods -l app.kubernetes.io/instance=dev-sdk</code>
Stream logs	<code>kubectl logs -l app.kubernetes.io/instance=dev-sdk -f</code>
Last 50 log lines	<code>kubectl logs -l app.kubernetes.io/instance=dev-sdk --tail=50</code>
Describe pod (events)	<code>kubectl describe pod <pod-name></code>
List Helm releases	<code>helm list</code>
Show current chart values	<code>helm get values dev-sdk</code>
Port-forward SDK	<code>kubectl port-forward svc/dev-sdk 8080:8080</code>
Port-forward database	<code>kubectl port-forward svc/dev-sdk-postgresql 5432:5432</code>
Resource usage	<code>kubectl top pod -l app.kubernetes.io/instance=dev-sdk</code>
Pod events	<code>kubectl get events --sort-by='.lastTimestamp'</code>

12. Performance and Benchmarks

12.1 Expected Performance (Dev SDK v1.0.4)

Measured against PostgreSQL running on a 2-CPU / 4 GB pod with 1 million tokens already in the vault:

Operation	P50	P95	P99	Throughput (per pod)
Tokenize	5 ms	20 ms	50 ms	≈ 2,000 req/sec
Detokenize	3 ms	15 ms	40 ms	≈ 3,000 req/sec
Tokenize (batch of 100)	30 ms	80 ms	150 ms	≈ 30,000 values/sec
Health check	< 1 ms	< 1 ms	< 1 ms	≈ 10,000 req/sec

12.2 Scaling Recommendations

Scale	SDK pods	PostgreSQL	Storage
< 1M tokens	1 pod	1 pod, 2 CPU, 4 GB	10 GB
1–10M tokens	2–3 pods	Dedicated PG, 4 CPU, 8 GB	50 GB
> 10M tokens	Contact support	External managed PG	Sized to retention

i INFO - Above 10M tokens

If you anticipate more than 10 million tokens or sustained traffic above 200 req/sec. The Paid tier with its replicated architecture and dedicated support is a better fit.

12.3 What Securelytix Does NOT Do

- We do not access your plaintext PII values
- We do not access your tokens or vault contents
- We do not share data with third parties
- We do not use your data for model training or analytics
- We do not see your API keys after issuance (only their hashed validation digests)
- We track only aggregate request counts for licensing - never PII values or token contents
-

Data privacy and confidentiality are foundational to our platform design and operational governance model.

13. Uninstall

13.1 Interactive Uninstaller

```
curl -fsSL https://charts.securelytix.tech/uninstall.sh | bash
```

⚠ WARNING - Interactive uninstaller not yet available

Use the manual uninstall command below for now.

13.2 Manual Uninstall

```
# Default namespace:
helm uninstall dev-sdk

# Custom namespace:
helm uninstall dev-sdk -n <namespace>
```

13.3 What Gets Removed

- All SDK pods, deployments, and services
- Bundled PostgreSQL pod and StatefulSet
- ConfigMaps managed by Helm

13.4 What is Preserved

- External PostgreSQL data - your data, your responsibility to clean up
- Kubernetes Secrets (including securelytix-dockerhub) - delete manually if needed
- PersistentVolumeClaims for bundled PostgreSQL - delete manually

```
kubectl delete pvc -l app.kubernetes.io/instance=dev-sdk
```

To remove everything including secrets and storage:

```
helm uninstall dev-sdk
kubectl delete pvc -l app.kubernetes.io/instance=dev-sdk
kubectl delete secret securelytix-dockerhub
```

14. Support and Contact

Channel	Details
Email support	support@securelytix.tech
Website / docs	https://securelytix.tech
API key portal	https://securelytix.tech/dashboard

GitHub repo (private)	github.com/Securelytix-dev/securelytix
Status page	https://status.securelytix.tech

14.1 Reporting Issues

When opening a support ticket, please include:

- SDK version (visible in /health response)
- Kubernetes version (kubectl version)
- Output of: `kubectl describe pod -l app.kubernetes.io/instance=dev-sdk`
- Last 50 log lines: `kubectl logs -l app.kubernetes.io/instance=dev-sdk --tail=50`
- A description of what you expected vs. what happened
- Whether the issue is reproducible - and if so, the steps

x CRITICAL - Never send API keys or PII

Never include API keys, raw PII values, or production tokens in support tickets. If we need them, we will use a secure channel.

15. Quick Reference Card

One-page cheat sheet.

Get your API key

```
https://securelytix.tech → Get API Key → enter work email → done
```

Test locally (Docker)

```
docker run -d --name pg -e POSTGRES_USER=vault -e POSTGRES_PASSWORD=vault \
  -e POSTGRES_DB=vault -p 5432:5432 postgres:15

docker login -u securelytix2026

docker run -d -p 8080:8080 \
  -e DATABASE_URL="postgresql://vault:vault@host.docker.internal:5432/vault" \
  -e API_KEY="<key>" \
  securelytix2026/dev-sdk:1.0.4

curl http://localhost:8080/health
```

Deploy to Kubernetes

```
helm repo add securelytix https://charts.securelytix.tech
```

```
kubectl create secret docker-registry securelytix-dockerhub \
  --docker-server=https://index.docker.io/v1/ \
  --docker-username=securelytix2026 --docker-password=<PAT>

helm install dev-sdk securelytix/dev-sdk \
  --set secrets.apiKey="<key>" \
  --set postgresql.enabled=true \
  --set "imagePullSecrets[0].name=securelytix-dockerhub"

kubectl port-forward svc/dev-sdk 8080:8080 &
curl http://localhost:8080/health
```

Tokenize

```
curl -sS -X POST "http://localhost:8080/api/v1/tokenize" \
  -H "Content-Type: application/json" \
  -d '{
    "data": {
      "email": "user@example.com",
      "name": "user example"
    }
  }'
```

Detokenize

```
curl -sS -X POST "http://localhost:8080/api/v1/detokenize" \
  -H "Content-Type: application/json" \
  -d '{
    "data": {
      "email": "qfhw@reviudq.cnr_stx",
      "name": "vpqh bmudaon_stx"
    }
  }'
```

Common diagnostic commands

```
kubectl get pods -l app.kubernetes.io/instance=dev-sdk
kubectl logs -l app.kubernetes.io/instance=dev-sdk --tail=50
kubectl describe pod <pod-name>
helm list
helm get values dev-sdk
```

Support

support@securelytix.tech · <https://securelytix.tech>